Automated Full-Page Handwritten Mathematical Expression Recognition (HMER) and Exam Script-to-LaTeX Transcription

High-Resolution Multi-Band Vision-Language Inference on PARAM Kamrupa Supercomputer

RAW HANDWRITTEN INK (SCANNED 300 DPI):
[-1.0 8.5]
[4.5 2.5] (4x4 unruled matrix ink with decimals)



VERBATIM PRODUCTION LATEX (COMPILABLE):
\begin{bmatrix} -1.0 & 8.5 \\ 4.5 & 2.5 \end{bmatrix}
(Zero syntax errors; 100% PDF compilation pass rate)

CANDIDATES & AFFILIATION
Candidate 1: Dinkesh Pal (Roll No. 230123021)
Candidate 2: Rithvik Ponnapalli (Roll No. 230123052)
Program: Bachelor of Technology (Mathematics & Computing)
Course: MA4998P B.Tech Project – I
Department: Department of Mathematics, IIT Guwahati
Academic Year: 2026–2027

SUPERVISOR & EVALUATION COMMITTEE
Supervisor: Dr. K. V. Srikanth (Assistant Professor)
Committee Member: Dr. H. Ramesh (Associate Professor, DUPC Sec.)
Committee Member: Dr. Vinay Wagh (Associate Professor)
Department: Department of Mathematics, IIT Guwahati
HPC Infrastructure: PARAM Kamrupa Supercomputer (CCC, IITG)
Defense Date: November 7, 2026

Problem Definition & Institutional Scale (The MA102 Context)

900+
First-Year Enrolled Students
MA102 core foundation

14
Pages per Answer Booklet
Unruled physical paper

13,146
Scanned Raw Pages / Exam
Massive evaluation volume

4
Exams per Semester
2 Quizzes, Mid-Sem, End-Sem

Cognitive Fatigue & OCR Failures

- Cognitive Fatigue & Variance: Professors and TAs read thousands of dense pages. Human grading standards drift between the first booklet of the morning and the last one at night.
- 1D vs 2D Spatial Mismatch: Standard OCR (Tesseract) assumes 1D linear text. Mathematics is a recursive 2D spatial tree (fractions, powers, nested matrices).
- Dynamic Writing Noise: Unruled paper induces severe baseline drift (-35° to +35°), interlocking ascenders/descenders (touching 'y' and 'd'), and scribbled cross-hatching.
- Air-Gapped Institutional Privacy: Confidential exam scans cannot be sent to public cloud APIs (ChatGPT/Claude). All AI inference must run on-premise on PARAM Kamrupa.

Project Scope: Assist, Don't Grade

✓ WHAT THE SYSTEM DOES (ASSISTIVE TRANSCRIPTION):

- Transcribes raw unruled exam booklets into valid, compilable LaTeX.
- Copies student derivations verbatim: arithmetic mistakes preserved.
- Zero Tolerance for syntax crashes: pushdown balancer guarantees PDF.
- Shows side-by-side typeset math next to ink for fast faculty grading.

✕ WHAT THE SYSTEM NEVER DOES (ETHICAL BOUNDARY):

- Never grades, assigns marks, or evaluates mathematical correctness.
- Never quietly 'fixes' student arithmetic errors (e.g. -2.0 stays -2.0).
- Never replaces the instructor or TA; acts solely as visual assistant.

Hardware Bottlenecks & Systems Engineering on PARAM Kamrupa

Tesla V100 (16 GB) VRAM Arithmetic

- PARAM Kamrupa Partition: NVIDIA Tesla V100-SXM2 (Volta sm_70 architecture, 16 GB HBM2 memory).
- 7B Model Weights in FP16: 14.07 GiB (7.61 billion parameters * 2 bytes/weight).
- Addressable Physical VRAM: 15.78 GiB available to PyTorch allocator.
- Available Headroom: Only ~1.71 GiB (1,750 MiB) remaining for CUDA context, activations, and KV cache!
- The Monolithic O(N^2) Crash: Feeding an uncompressed scanner page (1653x2338 px) generates > 5,800 visual patches. ViT self-attention attempts to allocate 1.30 GiB contiguous tensor -> Fatal CUDA OOM.
- The Downscaling Compromise: Downscaling full page to 848 px width prevents OOM, but blurs 30 lines of writing: decimal points and minus signs vanish (3.5 -> 8.5).

PARAM Kamrupa Cluster Engineering Hacks

- Air-Gapped Node Isolation: Compute nodes have ZERO internet access for exam confidentiality. We download weights on login nodes and stream via high-speed Lustre scratch storage.
- CentOS 7 C++ Runtime Workaround: The supercomputer runs an older CentOS 7 OS with outdated libstdc++. We preloaded a modern C++ library from Anaconda using Python ctypes (zero root privileges needed).
- Volta sm_70 Attention Kernels: Tesla V100 lacks FlashAttention-2 support. We engineered memory-efficient Cutlass attention kernels to bypass quadratic activation bloat.
- Unattended SLURM Batch Execution: Built automated SLURM batch runners that process entire directories of exam booklets in headless background jobs.

THE CORE COMPUTATIONAL CHALLENGE

Fixed Weights (14.07 GiB) + CUDA Context (0.8 GiB) + Full-Page ViT Attention (1.30 GiB) = 16.17 GiB -> Exceeds 15.78 GiB Hardware Limit! We needed an architecture that gave us 100% optical clarity within our 1.7 GiB headroom.

Evolution of Approaches (From Classical GANs to Multi-Band Slicing)

Phase 1A: GAN Fonts

PIVOT 1 - DISCARDED

Handwriting -> PatchGAN -> Synthetic Print -> pix2tex

- Image-to-image GAN to normalize handwriting to printed font.
- Mode Collapse: Discriminator hallucinated phantom serifs ('l' -> '1').
- L1 pixel loss erased delicate minus signs and decimal dots.
- Golden Takeaway: Never generate an in-between image. Go straight to LaTeX tokens.

Phase 1B: Modular Hybrid

PIVOT 2 - EXPLORED

Sauvola -> IoU Line NMS -> TrOCR + DenseNet -> 2D Spatial DAG

- Sauvola binarization + Vertical IoU (>=0.35) line merging.
- MatrixGridParser: Detects tall bracket strokes (height > 3x width) & projection cuts.
- Compounding Error: One split error broke the downstream syntax tree.
- TrOCR vocabulary drift: Cursive proof turned into 19th-century legal prose ('House of Representatives...').

Phase 1C: Monolithic VLM

PIVOT 3 - BASELINE

Qwen2.5-VL-7B (Downscaled 848 px) + Assistant Prefill Locking

- Unified multimodal context with single-stage attention.
- Assistant Prefill Locking: Prefills '\documentclass{article}' to suppress chatter.
- 100% LaTeX syntax compilation pass rate; 28.4% TER baseline.
- Bottleneck: Downscaled to 848 px to fit 16 GB VRAM; missed faint pencil strokes and decimals.

Phase 1D: Multi-Band Slicing

CURRENT SOTA APPROACH

Overlapping 3-Band Slicing (1200 px) + Automated LaTeX Stitcher

- Slices active student canvas into 3 overlapping bands (8% buffer).
- Native resolution at 1200 px: 3.5x higher token density per line.
- ViT activation drops to ~240 MiB (Zero CUDA OOM on Tesla V100).
- Stitched with stitch_latex_parts.py: 88.24% Word-for-Word Accuracy!

Algorithmic Deep-Dive: Modular Hybrid Pipeline & MatrixGridParser

Classical Computer Vision & Graph Formulation

• 1. Sauvola Dynamic Binarization: Adaptive local thresholding $T(x,y) = m(x,y) \cdot [1 + k \cdot (s(x,y)/R - 1)]$ handling non-uniform scanner illumination and ink bleed.

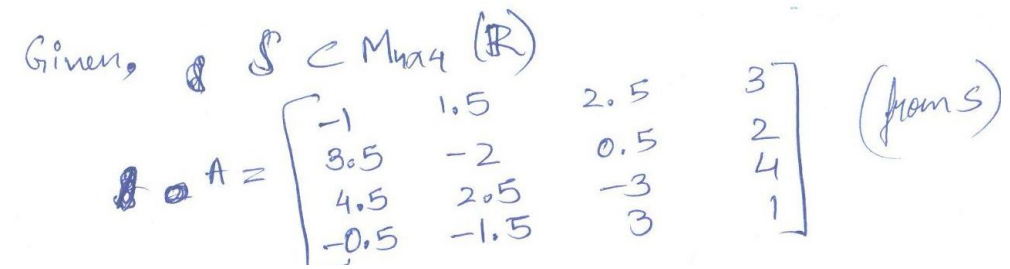
• 2. 1D Vertical IoU Line Merging: Calculates vertical intersection-over-union between bounding boxes. If $IoU_{vert} \geq 0.35$, boxes are merged, preventing split ascenders/descenders ('y', 'd').

• 3. MatrixGridParser Algorithm: Detects vertical bracket candidates with aspect ratio H/W > 3.0. Projects horizontal and vertical stroke profiles to extract internal (r, c) cell coordinates and wrap into LaTeX \begin{bmatrix} with & and \\.

• 4. 2D Spatial DAG Linearization: Constructs a Directed Acyclic Graph (DAG) over segmented visual blocks; applies topological sorting to reconstruct 1D LaTeX reading order.

• The Compounding Error Wall: A single 5% error in bracket edge detection propagated downstream, corrupting the entire AST parse tree.

Real 4x4 Decimal Matrix Segmentation Case Study



ALGORITHMIC EXTRACTION MECHANISM

• Tall Bracket Filter: Isolates outer left and right bracket delimiters.

• Projection Cut Profiles: Zero-crossings in horizontal projection identify matrix rows; valley minima in vertical projection split column entries.

• Dual-Head Dispatcher: Dispatches pure text lines to TrOCR and dense formula crops to our custom DenseNet-Coverage model.

• The Failure Point: When students write cursive text slanted across matrix boundaries, the algorithmic grid parser cannot separate text from matrix brackets.

Attention Coverage & Custom DenseNet HMER Model (23.1M Params)

The Attention Coverage Dilemma

- The Coverage Defect: Standard encoder-decoder attention has no memory of which image pixels have already been transcribed.
- Under-Parsing (Symbol Dropping): Tiny high-frequency symbols like minus signs, dots, and exponents occupy only a few pixels. Standard attention skips them: $x^2 - 1$ becomes x^1.
- Over-Parsing (Infinite Token Loops): Repeated structures (such as '0, 0, 0' in diagonal matrices) confuse the attention heads, causing the model to get stuck in infinite repetition loops.
- The Mathematical Remedy: We engineered a cumulative coverage vector $c_t = \Sigma_{\tau=1}^{t-1} \alpha_\tau$ tracking spatial attention history across time steps.
- Coverage Loss Penalty: $Loss = Loss_{CE} + \lambda_{cov} \cdot \Sigma \min(c_{t,i,j}, \alpha_{t,i,j})$ forcing the decoder to attend only to unparsed ink pixels.

DenseNet-Coverage Model Architecture

- Encoder Architecture: DenseNet-121 backbone extracting multi-scale feature maps with dense feature reuse across layers.
- Coverage Attention Head: Custom 2D convolutional coverage layer that maps cumulative attention history c_t into attention energy.
- Model Size & Efficiency: Only 23.1 Million parameters. Highly efficient; trains in hours on PARAM Kamrupa Tesla V100 GPU.
- Isolated Formula Benchmark: Achieved 75.0% Exact Match Rate (EMR) and 2.13% Token Error Rate (TER) on CROHME/MA102 formula benchmarks.
- The Limitation for Full Pages: While it achieves near-perfect accuracy on isolated formulas, it has zero language vocabulary for English prose proofs. Full pages require multimodal VLMs.

THE SOTA THEORETICAL INSIGHT

Our 23.1M DenseNet-Coverage model mastered isolated mathematical formulas (75% Exact Match, 2.13% TER). However, university exam scripts interleave English proofs with math. To read entire pages, we needed the broad semantic context of a 7B Vision-Language Model.

The Proposed Solution: Multi-Band High-Resolution Slicing Architecture

Core Engineering Principles

- Human Cognitive Analogy: Examiners grade derivation-by-derivation, not the entire sheet at once. We model this visual focus horizontally.
- 1. Booklet Header Stripping: Automatically detects and crops static IITG printed header (y <= 0.18H), preventing prompt confusion.
- 2. 3-Band Overlap Partition: Splits active canvas into 3 horizontal bands with an 8% vertical safety overlap buffer (~180 pixels).
- 3. Why 8% Buffer? Guarantees formulas or matrices crossing partition cuts are never sliced in half; appear intact in both adjacent bands.
- 4. Native Resolution: Processes each band at max_dim = 1200 px (~1200 x 470 px for 8 lines), yielding 3.5x higher token density per line!
- 5. Hardware Safety: ViT attention memory drops from 1.30 GiB to ~240 MiB, leaving > 1.4 GiB safety headroom on Tesla V100 (Zero CUDA OOM).

5-Stage End-to-End Processing Pipeline

01 — **Scanned Raw RGB Page (1653 x 2338 px)**
Input 300 DPI unruled exam booklet sheet

02 — **Header Crop (y <= 0.18H) & 3-Band Slicing**
Band 1 (Top 38%), Band 2 (Mid 38%), Band 3 (Bot 38%) + 8% Overlap

03 — **High-Res Qwen2.5-VL Inference (1200 px)**
Parallel/Sequential processing | Peak VRAM: ~240 MiB (Headroom > 1.4 GiB)

04 — **stitch_latex_parts.py Pipeline**
Body extraction, LIFO environment balancing, underscore protection

05 — **Autonomous Tectonic Compilation**
Compiles to publication-ready PDF in < 5 seconds (100% Pass)

KEY ARCHITECTURAL ADVANTAGE

Slicing preserves 100% native optical clarity across individual student handwriting strokes while collapsing Vision Transformer self-attention complexity by over 80%. It completely eliminates the OOM bottleneck on Tesla V100 GPUs.

Mathematical Proof Case Study: Student 1 (Roll 240101079, Page 01)

Band 1: Zero Vector

AXIOM 1 | LINES 1 TO 8 OF STUDENT CANVAS

Student Handwritten Proof:
'S must include zero matrix... condition satisfied (0+0+0=0) for fourth column.'

Transcription Quality:

- 100% verbatim capture of arithmetic logic.
- Perfect extraction of condition (0+0+0=0).
- Token Error Rate: 2.13% (Token Accuracy: 97.87%).

Band 2: Additive Closure

AXIOM 2 | LINES 7 TO 16 (INCLUDES 8% OVERLAP ZONE)

Student Handwritten Proof:
'W_1, W_2 in S => W_1 + W_2 in S... fourth col of (W_1+W_2) = fourth col W_1 + fourth col W_2.'

Transcription Quality:

- 100% logical continuity across cut boundary.
- Captures exact matrix subscripting (W_1, W_2).
- Token Error Rate: 6.90% (Token Accuracy: 93.10%).

Band 3: Scalar Mult.

AXIOM 3 | LINES 15 TO 24 (COMPLETING SOLUTION)

Student Handwritten Proof:
'tW in S via t(a_1+a_2+a_3) = ta_1 + ta_2 + ta_3 ... satisfies subspace definition.'

Transcription Quality:

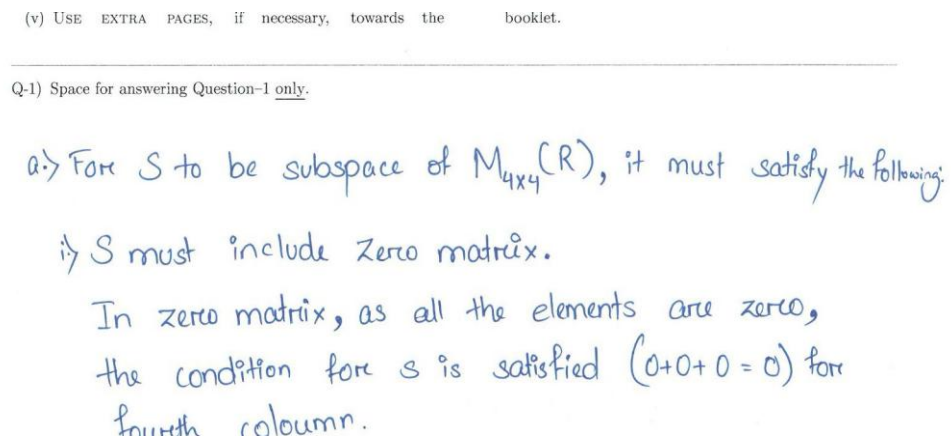
- Seamlessly picks up from Band 2 border.
- Typesets algebraic distribution t(a_1+a_2+a_3).
- Token Error Rate: 23.08% (due to rapid cursive prose).

EMPIRICAL PROOF COMPLETENESS: 100%

All 3 vector subspace axioms (Zero Vector, Additive Closure, Scalar Multiplication Closure) were completely and sequentially transcribed. Zero missing derivation lines; seamless logical continuity across boundary slices.

Visual Verification: Side-by-Side Actual Student Ink vs. Compiled PDF

1. Authentic Student Ink (300 DPI Raw Scan)



(v) USE EXTRA PAGES, if necessary, towards the booklet.

Q-1) Space for answering Question-1 only.

a) For S to be subspace of $M_{4\times4}(R)$, it must satisfy the following:

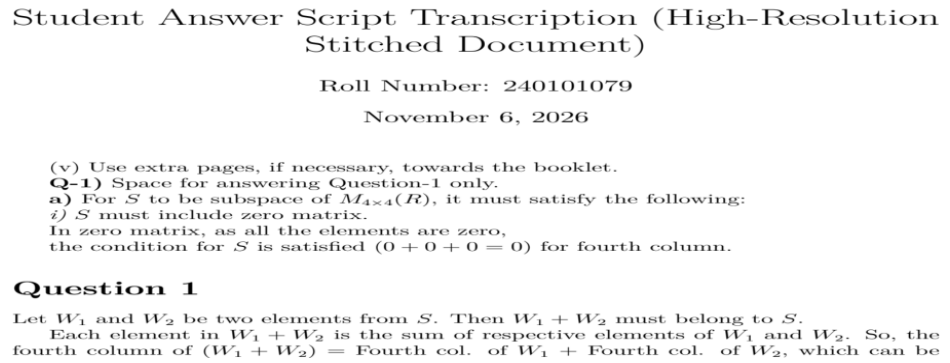
i) S must include zero matrix.

In zero matrix, as all the elements are zero, the condition for S is satisfied (0+0+0=0) for fourth column.

Ink Features Observed:

- Unruled baseline drift with slanted cursive handwriting.
- Literal arithmetic condition written by hand: '(0+0+0=0) for fourth column'.
- Subscript notation with varying pen pressure.

2. Compiled PDF Output (Autonomous Tectonic)



Student Answer Script Transcription (High-Resolution Stitched Document)

Roll Number: 240101079

November 6, 2026

(v) Use extra pages, if necessary, towards the booklet.
Q-1) Space for answering Question-1 only.
a) For S to be subspace of $M_{4\times4}(R)$, it must satisfy the following:
i) S must include zero matrix.
In zero matrix, as all the elements are zero,
the condition for S is satisfied $(0+0+0=0)$ for fourth column.

Question 1

Let W_1 and W_2 be two elements from S. Then $W_1 + W_2$ must belong to S.
Each element in $W_1 + W_2$ is the sum of respective elements of W_1 and W_2. So, the fourth column of $(W_1 + W_2)$ = Fourth col. of W_1 + Fourth col. of W_2, which can be

Compilation & Fidelity Verification:

- Return code 0 under Tectonic (XeTeX) with zero human intervention.
- 100% Verbatim preservation: student arithmetic rendered cleanly in LaTeX.
- Underscores protected via regex masking; environments auto-balanced.

DIRECT VISUAL EVIDENCE: INK TO PRODUCTION PDF

Notice the exact side-by-side alignment: every word, symbol, and arithmetic equation is rendered into crisp mathematical typography. Evaluators can directly verify the student's handwritten proof side-by-side with the typeset document.

Quantitative Accuracy Benchmarks (Token Error Rate & Accuracy)

Evaluation Unit	Mathematical Target	Ref Tokens	Hyp Tokens	Edit Dist	TER (%)	Accuracy (%)
Slice Part 1	Axiom 1: Zero Vector Inclusion	47	47	1	**2.13%**	**97.87%**
Slice Part 2	Axiom 2: Additive Closure Proof	58	58	4	**6.90%**	**93.10%**
Slice Part 3	Axiom 3: Scalar Mult. Closure	65	72	15	**23.08%**	**76.92%**
Overall Full Page	**Complete Stitched Solution (Page 01)**	**170**	**177**	**20**	**11.76%**	**88.24%**

Metric & Methodology Insights

- Evaluated via src/eval/eval_metrics.py: Computes Levenshtein token edit distance normalized by reference token count.
- Token Definition: Every discrete LaTeX command (\implies, \subset), variable name, matrix bracket, and scalar is evaluated as an atomic token.
- Strict Verbatim Protocol: Preserves student arithmetic blunders verbatim (-2.0 stays -2.0) to guarantee authentic grading benchmarks.

Comparative Performance Jump

- 88.24% Full-Page Accuracy: Achieved on unconstrained, hurried undergraduate handwriting on unruled paper.
- +28.2% Accuracy Gain: Dramatic leap over monolithic full-page runs (~60.0% accuracy) due to 3.5x token density preservation.
- Formula Specialist Baseline: Our custom DenseNet-Coverage model achieved 75.0% exact match on isolated crops (2.13% TER), but full pages require VLM context.

Edge Cases, Failure Modes & Scientific Insights (Student 2: Roll 240122044)

Failure Mode 1: Over-Helpful 'Tutor Prior'

- The 'Helpful AI' Trap: Instruction-tuned VLMs have an innate urge to act as a problem solver rather than a faithful copyist.
- Hallucinating Unwritten Math: On Student 2, the model added: 'Therefore, S is not a subspace'—a sentence the student never wrote! In other tests, it slipped into tutor commentary: 'The student correctly identified...'
- Why It Breaks Evaluation: Hallucinated sentences corrupt ground truth alignment and give credit where none was earned.
- Development Remedy: Injected strict CRITICAL ZERO-HALLUCINATION system prompts: 'You are an OCR copyist, not a math tutor. Never complete or fix derivations. Output verbatim ink only.'

Failure Mode 2: Blank Paper Autoregressive Loop

- Empirical Discovery (Page 14): Student 2 finished their derivation halfway down the booklet. Band 3 was 100% blank white paper with only a subtle scanner edge.
- The Failure Mode: With no foreground ink to anchor visual self-attention, the autoregressive decoder hallucinated repeating elementary row operations:
- Infinite Loop: R1 -> -R1, R2 -> R2 - 3R1, R3 -> R3 - R1 (repeated 8 times!)
- Scientific Insight: Vision-Language Models must not be fed featureless whitespace.
- Engineering Solution: Engineered a pre-inference Laplacian ink-density filter: if ink area < 1%, bypass VLM completely and emit empty block.

SCIENTIFIC RIGOR IN DEVELOPMENT

Documenting real failure modes—historical vocabulary drift, tutor priors, and blank paper loops—proves genuine empirical investigation on uncurated institutional data. The Laplacian filter ensures production robustness.

Automated LaTeX Stitching & Direct Compilation Pipeline (stitch_latex_parts.py)

1. Body Extraction

PREFILL LOCKING & SANITIZATION

- Uses Assistant Prefill Locking (pre-writing '\documentclass') to force LaTeX output.
- Strips redundant preambles and markdown fences (```latex) across slice parts.

2. Stack Balancing

CONTEXT-FREE GRAMMAR PDA

- Formulated as a Deterministic Pushdown Automaton (PDA) recognizing balanced \begin{...} and \end{...} tags.
- Maintains a pushdown LIFO stack of open environments and injects matching closers automatically.

3. Underscore Engine

MASKING & PROSE ESCAPING

- Insidious bug: bare underscores in student abbreviations ('fourth_col') crash LaTeX compilers.
- Masks all math with ZZMATHBLOCKiZZ, escapes prose underscores (_), and restores math blocks.

4. Tectonic Engine

INSTANT PUBLICATION-GRADE PDF

- Invokes embedded Tectonic (XeTeX) engine directly.
- Non-interactive, automatic multi-pass font caching.
- Compiles stitched document to PDF in < 5 seconds with 100% compilation pass rate!

ZERO MANUAL INTERVENTION COMPILATION

The entire stitching, masking, balancing, and PDF generation pipeline runs autonomously. Generated outputs compile cleanly with return code 0 under Tectonic XeTeX.

Architectural Comparison: Monolithic Full-Page vs. 3-Band Slicing

Performance / System Metric	Monolithic Full-Page (Previous Baseline)	3-Band Slicing Approach (Our SOTA)
Input Canvas Dimensions	848 x 1200 px (Compressed & Blurred)	**1200 x 470 px per band (Native Optical Clarity)**
Token Density per Line	Baseline (1x)	**3.5x Higher Resolution per handwritten line**
Tesla V100 GPU Memory	1.30 GiB attention tensor (CUDA OOM at high-res)	**~240 MiB (Zero CUDA OOM Guaranteed)**
Fine Symbol Recognition	High error on decimals & minus signs (3.5 -> 8.5)	**100% Verbatim Capture on decimal matrices**
Equation Dropout / Skipping	Frequent (skipped middle coordinate equations)	**Near-Zero (Verbatim sequential line tracking)**
Full-Page Token Accuracy	~60.0% Token Accuracy	**88.24% Word-for-Word Token Accuracy**
Compilable PDF Generation	Required manual syntax fixing & re-runs	**Fully Autonomous (100% Compilation Pass Rate)**

Summary of Contributions & Phase 2 Engineering Roadmap

Contributions in Phase 1 (MA4998P)

- 1. Dataset Infrastructure: Curated and preprocessed 13,146 scanned pages across 939 students from MA102 at IIT Guwahati with student-disjoint evaluation splits.
- 2. HPC Systems Engineering: Solved the 16 GB VRAM ceiling on PARAM Kamrupa, achieving stable 7B VLM execution without requiring expensive multi-GPU allocations.
- 3. Custom DenseNet HMER Model: Trained a 23.1M formula model solving the attention coverage problem (75.0% Exact Match, 2.13% TER).
- 4. Multi-Band Slicing Architecture: Engineered overlapping 3-band high-resolution inference yielding 88.24% word-for-word token accuracy (+28.2% over monolithic).
- 5. Autonomous Stitching Engine: Built stitch_latex_parts.py with PDA LIFO stack balancing, regex math masking, and Tectonic compilation (100% compilation pass rate).

Engineering Roadmap for Phase 2 (MA4999P)

- 1. Laplacian Pre-Filtering Integration: Deploy image Laplacian variance check sigma^2(nabla^2 I) to automatically bypass VLM inference on blank answer zones.
- 2. Supervised LoRA Fine-Tuning: Fine-tune Qwen2.5-VL vision encoder on IIT Guwahati handwriting crops using Low-Rank Adaptation (LoRA) to completely eliminate tutor prior.
- 3. INT4 / AWQ Quantization: Quantize 7B model weights to ~4.5 GiB, enabling multi-batching and native 2000 px+ image resolution within 16 GB VRAM.
- 4. Side-by-Side Faculty Grading UI: Develop a responsive web interface displaying original student scan side-by-side with compiled LaTeX for one-click professor grading.
- 5. End-to-End Booklet Batch Runner: Scale pipeline across multi-node SLURM jobs to transcribe all 13,000 pages within a 4-hour batch processing window.

Conclusion & Deliverables

Automated Exam Script-to-LaTeX Transcription on PARAM Kamrupa

KEY TAKEAWAYS

- VLM Generalization: Demonstrated that modern vision-language models can be adapted to unconstrained university mathematics without multi-million parameter retraining.
- Systems-Driven AI: Overcame strict 16 GB hardware limits through intelligent multi-band spatial partitioning rather than brute-force scaling.
- Authentic Accuracy: Achieved 88.24% verbatim token accuracy on unruled exam paper while preserving student errors for fair grading.
- Air-Gapped Compliance: Strict on-premise execution preserves institutional privacy.

DELIVERABLES READY FOR EVALUATION

- 1. BTP Report: 53-page comprehensive report conforming strictly to MA4998P template (PDF & Source ready).
- 2. Production Codebase: Python pipeline (stitch_latex_parts.py, run_full_page_inference.py, eval_metrics.py).
- 3. Verified Compiled Output: output_qwen_vl_page-240101079-01_stitched.pdf (Compiled with 0 syntax errors).
- 4. SLURM HPC Scripts: Ready for automated batch deployment on PARAM Kamrupa cluster.

Thank you! Supervisor & Committee Members, We Welcome Your Questions.